

8.7 模拟面试题

什么是转换构造？

可以发生隐式类型转换的构造函数

构造函数只有一个参数（不包括默认参数this），且非当前类类型的对象，可在用=初始化或拷贝赋值的时候将类型转为当前类类型的过程交隐式类型转换；

```
class CTest{
public:
    CTest(int a){} //转换构造
    CTest(int a, int b=0){} //转换构造

};

int main()
{
    CTest c1;
    c1 = 9; //拷贝赋值，将9强转为临时对象给c1赋值
    CTest c2 = 12; //初始化，隐式类型转换，将12转换为CTest，调用拷贝构造

    return 0;
}
```

构造函数发生隐式转换的情况：（不包括默认参数this）

- 1.真的只有只有一个参数
- 2.有多个参数，但是只有第一个参数没有给默认值，其他有默认值的参数会被编译器忽略

如果想要避免隐式转换可以使用 explicit关键字

inline

C++中inline编译限制：

1. 不能存在任何形式的循环语句
2. 不能存在过多的条件判断语句
3. 函数体不能过于庞大（针对“函数本体”所用开销比“函数调用”更多，）

具体场景：

4. 空构造和空析构不能inline(因为不是真的为空, 编译器会自动填充对应构造和析构的代码; 那么填充的代码中可能就会有inline的函数, 导致重复很多代码)
5. 虚函数不能inline, 因为虚函数是动态绑定的, 在运行期确定函数;

总之, 将 inline 限制在小型、被频繁调用的函数上;

如何查看编译器是否执行inline

查看汇编, 如果没有内联直接是函数调用 call; 否则就是代码拷贝执行;

```
#include<iostream>
using namespace std;
class CTest {
public:
    inline int func1() {
        int a = 1;
        return a;
    }
    inline void func2(bool flag) {
        if (flag) {
            for (int i = 0; i < 3; i++) {
                cout << i << ' ';
            }
            cout << '\n';
        } else {
            for (int i = 5; i < 7; i++) {
                cout << i << ' ';
            }
            cout << '\n';
        }
        return;
    }
};
```

```
CTest test;
test.func1();
0096237F lea     ecx,[test]
00962382 call    std::tuple<std::_F
    bool flag = true;
00962387 mov     byte ptr [flag],1
    test.func2(flag);
0096238B movzx   eax,byte ptr [fla
```

```

class CTest {
public:
    inline int func1() {
00E92300  push     ebp      已用时间 <= 1ms
00E92301  mov     ebp,esp
00E92303  sub     esp,0D8h
00E92309  push     ebx
00E9230A  push     esi
00E9230B  push     edi
00E9230C  push     ecx
00E9230D  lea     edi,[ebp-18h]
00E92310  mov     ecx,6
00E92315  mov     eax,0CCCCCCCCh
00E9231A  rep stos dword ptr es:[edi]
00E9231C  pop     ecx
00E9231D  mov     dword ptr [this],ecx
00E92320  mov     ecx,offset _A10D4642_main@cpp (0EA3029h)
00E92325  call    @_CheckForDebuggerJustMyCode@4 (0E9155Fh)

        int a = 1;
00E9232A  mov     dword ptr [a],1
        return a;
00E92331  mov     eax,dword ptr [a]
    }

```

```

0096238F  push     eax
00962390  lea     ecx,[test]
00962393  call    CTest::func2 (0961690h)    已用时间 <= 1ms
}

```

VS define

- 工作时期: inline函数是 `inline` 关键字定义; 编译时展开 `#define` 是预处理器指令定义; 预处理期展开
 - 类型检查: inline函数编译器会进行类型检查——参数类型是否匹配, 确保函数调用的正确性。可重载, 支持函数重载机制。宏没有类型检查。预处理器只是简单地进行文本替换, 不支持重载, 只能通过不同的宏名来区分。
 - 调试: inline函数的调试信息更丰富。编译器会生成调试符号, 调试器可以像调试普通函数一样调试内联函数。宏的调试信息较少。预处理器展开后, 调试器看到的是展开后的代码, 而不是宏的定义。